

Documento de Requisitos (PRD) - CazaChollos

Versión: 1.0 (Architecture-Ready)

Fecha: 2026-01-02

Estado: Aprobado para desarrollo

Stack: Python + Docker + PostgreSQL

1. Visión del Proyecto

Desarrollar un sistema escalable y contenerizado que monitorice grupos privados de Telegram, estructure la información de ofertas ("chollos") y permita su consulta a través de un Bot, además de proveer un panel visual para la monitorización de datos.

Principios de Diseño:

- **Zero-Duplication:** Lógica robusta para evitar procesar el mismo chollo repetido (spam).
- **Separation of Concerns:** Arquitectura de microservicios lógica vía Docker Compose.
- **Type Safety:** Uso estricto de tipos en Python.

2. Arquitectura Técnica

El sistema se orquestará mediante `docker-compose` con los siguientes servicios:

2.1 Stack Tecnológico

- **Lenguaje:** Python 3.11+
- **Gestión de Dependencias:** Poetry (o uv)
- **Base de Datos:** PostgreSQL 16 (Imagen `alpine`)
- **ORM:** SQLAlchemy 2.0 (Async) + Alembic (Migraciones)
- **Dashboard:** Streamlit

2.2 Servicios (Contenedores)

1. **db (PostgreSQL):**
 - Persistencia principal.
 - Volumen persistente para datos (`pgdata`).
 - Configuración optimizada para escritura y búsquedas FTS.
2. **ingestor (Service):**
 - Cliente de Telegram (User Client).
 - Escucha eventos, normaliza y escribe en `db`.
3. **bot (Service):**
 - Interfaz de usuario (Telegram Bot API).
 - Consulta la `db` (Read-only preferente) y responde al usuario.

4. **dashboard** (Web App):

- Panel de administración construido en **Streamlit**.
- Puerto expuesto: **8501**.
- Permite visualizar últimos chollos, estadísticas de ingesta y validación de parsers.

3. Modelo de Datos (PostgreSQL)

3.1 Tabla: **raw_messages**

Auditoría y trazabilidad. Se guarda todo antes de procesar.

- **msg_id** (PK compuesta con chat_id)
- **chat_id**
- **raw_content** (Text)
- **created_at** (Timestamp)

3.2 Tabla: **deals**

Entidad normalizada para búsquedas.

- **id**: UUID (PK).
- **content_hash**: **String (Unique Index)**. Hash MD5(url_limpia + precio + titulo_key) para deduplicación estricta.
- **title**: String.
- **description**: Text.
- **price_sale**: Numeric (Decimal exacto).
- **price_retail**: Numeric (Nullable).
- **currency**: String (Default "EUR").
- **category**: Enum (tecnología, hogar, cocina, etc.).
- **url**: String (Saneada, sin trackers).
- **chollo_score**: Integer (0-100).
- **search_vector**: **TSVECTOR** (Generado auto para búsqueda semántica/full-text).
- **created_at**: Timestamp.

4. Requisitos Funcionales por Servicio

4.1 Servicio Ingestor (The Listener)

1. **Conexión Persistente**: Debe mantener la sesión de Telegram (**.session**) en un volumen para evitar re-login.
2. **Deduplicación**:
 - Al recibir mensaje $\rightarrow$ Calcular **content_hash**.
 - Si existe en DB $\rightarrow$ Ignorar (o actualizar **last_seen**).
 - Si NO existe $\rightarrow$ Procesar e Insertar.
3. **Parsing**:
 - Uso de librería **price-parser**.
 - Limpieza de URL (eliminar parámetros **utm_source**, etc.).

- Categorización basada en palabras clave.

4.2 Servicio Bot (The Interface)

1. **Búsqueda Natural:**
 - Input: "Auriculares sony baratos".
 - Proceso: Convertir a `plainto_tsquery` de Postgres + Filtros SQL.
2. **Ranking:** Ordenar por una combinación ponderada de **Relevancia de Texto** y **Chollo Score**.
3. **Respuesta:** Mostrar Top 5 resultados con formato limpio (HTML/Markdown).

4.3 Servicio Dashboard (Web Admin)

1. **KPIs en tiempo real:**
 - Total de ofertas ingestadas hoy.
 - Ofertas por hora (para detectar picos).
2. **Explorador de Datos:**
 - Tabla paginada con los últimos **deals** insertados.
 - Columnas: Título, Precio, Score, Categoría, Link original.
3. **Debug Visual:**
 - Histograma de distribución de **chollo_score** (para calibrar si somos muy estrictos o laxos).

5. Algoritmo "Chollo Score" (Lógica de Negocio)

Fórmula inicial para puntuar la calidad de la oferta (0-100):

$Score = (Descuento_{\{pct\}} \times 0.7) + Bonus - Penalizaciones$

- **Bonus:**
 - **+10** si existe Precio Real fiable (detectado como "ANTES" o "PVP").
 - **+5** si tienda es "Tier 1" (Amazon, etc.).
- **Penalizaciones:**
 - Decaimiento temporal: El score baja conforme pasa el tiempo desde `created_at`.
- **Límite:** El resultado se corta (clamp) entre 0 y 100.

6. Estructura del Proyecto

Diseño modular para facilitar mantenimiento y escalabilidad.

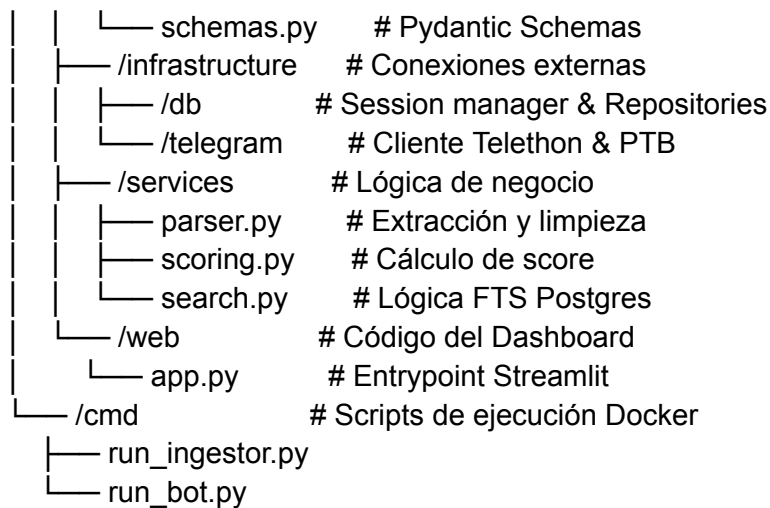
Plaintext

/cazachollos

```

├── docker-compose.yml    # Orquestación (db, ingestor, bot, dashboard)
├── .env.example
├── pyproject.toml        # Dependencias (Poetry)
├── /src
│   ├── /domain           # Modelos puros y esquemas
│   └── models.py         # SQLAlchemy Models

```



7. Hoja de Ruta (Roadmap) MVP

1. **Infraestructura:** Crear [docker-compose.yml](#) y levantar Postgres + Streamlit "Hola Mundo".
2. **Modelado:** Definir tablas en SQLAlchemy y ejecutar migración inicial con Alembic.
3. **Ingesta:** Programar el cliente de Telegram y validar que guarda datos en DB.
4. **Visualización:** Conectar Streamlit a la DB para ver las filas entrando en tiempo real.
5. **Consumo:** Programar la búsqueda FTS y conectar el Bot.